

2026-08-24 更新:ColBERT 检索器替换实验——验证 gate 的可迁移性

一、动机

之前所有实验都固定用 BAAI/bge-base-en-v1.5(双塔 + cosine)做候选检索,gate 一直是接在这一个检索器后面调出来的。这次想验证一个跟"继续调 gate 本身"不同的问题:**gate 带来的收益是不是绑定在 BGE 这一个检索器上的,换一个架构完全不同的检索器,gate 还有没有用**。选择 ColBERT(late interaction / MaxSim,逐 token 精细匹配,而不是 BGE 那种整句压缩成一个向量再算 cosine)作为对照,因为它跟 BGE 的检索范式差异足够大。

二、接入方式

retrieval/run_retrieval_exp.py 里已经有一个真正干活的 colbert_retrieval() 函数(用官方 colbert-ai 库,in-memory MaxSim,不需要离线建索引——每道题自己的候选池本来就只有 10-20 段,不是从海量共享语料库里检索),但从未被真正跑过、也没有环境装这个包。这次:

1. pip install colbert-ai(msys 环境此前没装),colbertv2.0 checkpoint 本地已有缓存。
2. run_retrieval_exp_wavefront_gate_v3_rawprefix.py(D 组主力脚本)新增 --retriever {cosine,colbert} 开关,默认仍是 cosine,不影响现有行为;新增 --colbert-model 参数。BGE 的检索指令前缀(--query-instruction)是 BGE 专用的非对称检索约定,ColBERT 训练时没见过,已确认只在 cosine 分支生效,不会错误地套到 ColBERT 查询上。

三、发现的问题:ColBERT 分数和 gate_score 量级不兼容,直接加权会让 gate 失效

现有的重排序公式是 $final_score = emb_score - \lambda \cdot gate_score$, $\lambda=0.50$ 是专门针对 BGE cosine 分数(范围大致 0-1)调出来的。ColBERT 的 MaxSim 分数是逐 query token 取最大值再加总得到的,不仅数值范围完全不同(实测量级在个位数到二三十不等),而且同一个量级本身也不可比——query 越长,加总出来的分数天然越大,不像 cosine 有个统一的上下界。如果直接把 ColBERT 分数代入原公式, $\lambda \cdot gate_score$ (最大也就 0.5)相对 ColBERT 分数(十几到二十几)小到可以忽略, gate 这个信号会被架空、基本不影响排序。

修复:新增 minmax_normalize_candidates(),只在 --retriever colbert 分支生效——对每一跳自己的候选池(比如 10 个候选)内部做 min-max 归一化,缩放到 0-1 区间,再代入 $rank_key = emb_score - \lambda \cdot gate_score$ 。这样不管某道题的 ColBERT 原始分数量级多大,归一化后都跟 gate_score 在同一个尺度上, λ 才有意义。cosine 分支的原始分数不受影响。

四、实验设计与结果

三个数据集,每个数据集两组对照(其余配置不变:gate v3 融合、rawprefix、 $\lambda=0.50$ 用于"加 gate"这组、comparison_hint reader prompt):

- 无 gate: --lambda-gate 0.0,公式退化成纯 ColBERT 排序,不受归一化影响(min-max 是保序变换, $\lambda=0$ 时排序结果跟用不用归一化完全一样)。
- 加 gate: --lambda-gate 0.50(沿用 BGE 那边调出来的值,没有专门为 ColBERT 重新扫,只是一个合理起点)。

数据集	方案	recall@1	recall@3	chain@1	EM	F1
MuSiQue	ColBERT 无 gate	0.6175	0.7577	0.3732	0.3322	0.4163
	ColBERT + gate	0.6581	0.7987	0.4348	0.3839	0.4734
	BGE + gate (现有最佳)	0.7018	0.8441	0.5143	0.4427	0.5386
2WikiMultihopQA	ColBERT 无 gate	0.8413	0.9167	0.6802	0.5223	0.5881
	ColBERT + gate	0.8742	0.9600	0.7454	0.5496	0.6192
	BGE + gate (现有最佳)	0.9289	0.9839	0.8613	0.5982	0.6770
HotpotQA	ColBERT 无 gate	0.5606	0.7468	0.3685	0.4623	0.5817
	ColBERT + gate	0.6027	0.8125	0.4498	0.5136	0.6408
	BGE + gate (现有最佳)	0.6595	0.8683	0.5567	0.5517	0.6870

五、结论

1. 归一化修复是有效的，gate 在 ColBERT 上依然有真实提升：三个数据集上"加不加 gate"这个对照，ColBERT 加了 gate 后 recall@1 涨 3-4 个点、chain@1 涨 6-8 个点、EM 涨 4-5 个点，跟 BGE 那边加 gate 的提升量级相当。说明之前"直接加权可能不行"这个担心，靠 min-max 归一化解决了，gate 没有被架空。

2. 但换 ColBERT 本身不划算：不管加不加 gate，ColBERT 三个数据集上全面弱于现有的 BGE (recall@1 差 4-6 个点，EM 差 4-6 个点，方向一致)。这符合预期——[colbert-ir/colbertv2.0](#) 是 2021 年、主要在 MS MARCO 上训练的模型，[BAAI/bge-base-en-v1.5](#) 是更新、覆盖面更广的通用 向量模型；ColBERT"逐 token 精细匹配"的架构优势没能弥补训练数据和时效性上的劣势。

3. 最有价值的发现是 gate 的可迁移性：同一个 gate (v3 融合信号)、同一套 rerank 公式，接在两个架构完全不同的检索器 (BGE 双塔 cosine、ColBERT late interaction) 后面，都带来了同样 量级的提升——说明 gate 提供的增量价值**不依赖具体检索器**，是一个可以接在任意打分范围的检索器后面的通用"插件"，只要做好分数归一化。这是第一次跳出 BGE 这一个检索方案去验证 gate 本身的 可迁移性，而不是又一次在同一个检索器上调参。

结论：不建议把 ColBERT 换成默认检索器 (BGE 仍然更好)，这条探索到此为止；除非以后想换一个比 [colbertv2.0](#) 更新的 late-interaction 模型再试，暂不计划继续在 ColBERT 自己的 λ 上投入 (比如单独为它扫一遍最优 λ)。

六、附：在线增量 listwise gate 探索 (新方向尝试，目前未跑赢基线)

同一天里还试了另一条完全不同的新方向，跟上面的 ColBERT 无关，记在一起方便回顾。

6.1 动机与设计

现有 gate v2/v3 都是"先收集数据、再一次性训练"的离线流程，训练时用的是 gold 中间答案展开 [Answer N]，跟推理时用模型自己（可能不准）的中间答案展开存在天然的分布差异。这次想验证一个不同的训练范式：**DAgger 式在线增量训练**——不是收集完数据再训练，而是在单次遍历数据集的过程中边训边用：

- 每一跳都用**当前已训练好的模型自己的真实选择**去展开下一跳的 [Answer N]（而不是 gold 答案），这样训练时看到的上下文分布跟推理时更接近。
- 打分用 listwise softmax 交叉熵（ListNet 式）而不是独立二分类：同一跳的候选池内部排序，gold 候选的 index 作为 softmax 目标。
- 特征上不再把 query/passage 相似度预先算成一个 cosine 标量喂给模型，而是把 BGE query 向量、BGE passage 向量、Llama h_after、 Δh 四路**各自独立 PCA** 后拼接，让模型自己学怎么比较 query 和 passage——但纯线性打分学不出这种"比较"（向量点积是双线性的，线性层学不到），所以最终打分公式是 $score = q^T W d + w_h^T h + w_{\Delta}^T \Delta h + b$ ，对 (q, d) 显式加了一个双线性交叉项，h/ Δh 保持纯线性（不需要跟别的向量比较）。
- **只有一个模型**，不像 gate v2/v3 按 (K, j) 分开池化——h_after 本身已经是"整条历史累积后"的表征，不需要按 hop 位置单独建模。
- **经验池只增不减、增量重训**：每一跳把新数据加入累积池后，用全部累积数据（不是只用这一跳的新数据）重新训练一遍，避免后面 hop 训练把前面 hop 学到的能力冲掉。
- 训练过程只能用 **GT decompose**（MuSiQue-only），因为每一跳需要知道 gold 是不是在候选池里、才能算 listwise loss——这跟 gate v2/v3 当初训练用 GT 的约束一致。

新建了 online_listwise_gate/ 目录，两个脚本：train_online_wavefront.py（在线训练，边训边用真实决策推进 trace）、run_wavefront_online_gate.py（训练完冻结模型后，做一次干净的、纯推理的 wavefront，这才是真正要看的评测结果，而不是训练过程中"边训边选"阶段的中间指标）。

6.2 数据增强：混入之前做过的 K=3/4 改写数据

MuSiQue train 的 GT decompose 里 K 分布很不均衡（2-hop 14376 / 3-hop 4387 / 4-hop 1175，K=2 占 72%）。想起来之前用 traces/enhance_decompose_nl.py 对 K=3/4 做过一轮 LLM 改写增强（train_nl_enhance.jsonl，5562 条改写、verify_pass 过滤后剩 4132 条可用），这次全量训练把它接了进去（train_online_wavefront.py 在 --split train 时会自动探测同目录下的 _enhance.jsonl 并合并，按 source_id 映射回原始段落/gold evidence，主问题文本也用改写后的版本以保持跟改写后的子问题链一致）。混入后 K 分布变成 2-hop 14376 / 3-hop 7671 / 4-hop 2023，3/4-hop 样本量接近翻倍。

6.3 全量训练结果（过程指标，不是最终评测）

全量 MuSiQue train（含增强数据，共 24070 条），单卡跑了 4.43 小时。每一跳"模型自己实时选择 vs gold"的准确率（训练过程指标，不是最终效果）：

hop	n_active	pick_accuracy
1	24070	0.9025
2	24070	0.8168
3	9694	0.7376
4	2023	0.7879

hop1→hop3 逐步下降符合预期（误差随累积上下文传播，[Answer N] 展开用的是模型自己不完美的前序预测）；hop4 略有回升，原因不明（可能是增强数据让这个桶的分布跟 hop3 不太一样），没有进一步深挖。

6.4 冻结模型评测：一个容易踩的坑——GT vs BART decompose 不能直接比

用冻结模型在 MuSiQue dev（2417 条）上跑了两次干净 wavefront，一次 `--decompose-mode gt`，一次 `--decompose-mode bart_decompose`：

decompose 模式	recall@1	recall@3	chain@1	EM	F1
GT decompose	0.7823	0.9210	0.6074	0.4791	0.5806
BART decompose	0.6709	0.8226	0.4919	0.4336	0.5278
gate v3 D 组（现有最佳，BART decompose）	0.7018	0.8441	0.5143	0.4427	0.5386

第一次看 GT decompose 那组数字（比 D 组全指标高 3.6~9.3 个百分点）一度以为是很大的提升，但这个对比是无效的——**gate v3 D 组的基准数字本身是 BART decompose 跑出来的**（D 组的定位就是用真实推理条件下能拿到的 BART 解耦做端到端评测），GT decompose 的子问题本身就好比 BART 生成的干净得多，这个差距（Appendix A.2 记录过的 GT-vs-BART gap）跟 gate 好不好无关，会直接拉高所有指标。换成同样用 BART decompose 的公平对比后，新方法在全部 5 个指标上都比 **gate v3 现有方法差**（recall@1 -3.09pp、recall@3 -2.15pp、chain@1 -2.24pp、EM -0.91pp、F1 -1.08pp）。

6.5 结论

这条新方向目前没有跑赢现有 **gate v3 方法**，公平对比（同用 BART decompose）下全指标落后。猜测（未验证）原因：

- 1. 训练只见过 GT decompose 的干净 query，评测换成 BART 的嘈杂 query 后有分布偏移——这是这个新方法特有的问题，不是 GT-vs-BART gap 本身；gate v3 打分基于 Llama hidden state（对措辞变化可能更鲁棒），这里额外直接把原始 BGE query 向量（经 PCA）喂进去，更直接暴露在 query 文本质量差异下。
- 2. 四路 PCA 只在 hop1 的 GT-decompose 数据上拟合过一次，BART query 的分布如果跟 GT 不一致，这个投影基未必还适用。
- 3. 完全没调过超参（`lr=0.05`、`pca_dim=64`、`train_epochs_per_hop=30` 都是随手定的默认值），跟 gate v3 那套反复调过的 `λ/rawprefix/comparison_hint` 不是同一个成熟度级别的对比。

方法本身增加了不小的复杂度（四路独立 PCA、双线性交叉项、在线增量训练、经验池累积重训），目前换来的是比现有简单 fusion gate 更差的效果，尚未证明这个方向的复杂度是值得的。是否继续投入（比如先定量看 BART query 到底跟 GT 差多少、检查 PCA 基是否失配）还是搁置，待定。